

Last updated on **May 6, 2026**

Introduction to generative AI apps on Databricks

Databricks supports both simple and complex GenAI applications, from Retrieval Augmented Generation (RAG) chatbots to tool-calling agents. These guides explain key concepts and development guides for key scenarios.

Learn GenAI concepts

Get familiar with foundational GenAI concepts, such as models, agents, tools, and apps.

Concepts: Generative AI on Databricks

Learn about GenAI models, agents, tools, and apps.

Databricks generative AI capabilities

Learn about all the GenAI capabilities on Databricks

Key challenges in building GenAI apps

Learn about key challenges of GenAI and how Databricks addresses them.

Agent system design patterns

Learn about options and trade-offs for agent designs, from simple chains to complex multi-agent systems.

Development guides

 Ask Genie

Learn about the development workflow for GenAI applications.

Guide: Agents development workflow

Understand the full lifecycle of building an AI agent.

Guide: RAG (Retrieval Augmented Generation) development

Understand the full lifecycle of building an RAG system.



Is this page

as this page helpful?



Yes



No



Send feedback



Ask Genie

Last updated on **Mar 27, 2026**

Concepts: Generative AI on Databricks

A GenAI app is an application that uses generative AI models (such as large language models, image generation models, and text-to-speech models) to create new outputs, automate complex tasks, or engage in intelligent interactions based on user input.

A GenAI app can be powered by simple calls to LLMs or other GenAI models, or by complex AI agents. Read more about [levels of complexity](#).

Agents, tools, evaluation, models, and other aspects of GenAI apps can be customized with your proprietary data. This data-driven customization leads to [data intelligence](#), allowing you to go beyond the general intelligence offered by canned AI models.

GenAI applications

A user-facing GenAI application can take many forms, such as:

- A chat app, such as one deployed using [Databricks Apps](#)
- An API endpoint, such as an [agent deployed to Model Serving](#)
- A SQL function for analysts, such as an [AI Function](#)

Success with GenAI applications often requires two sets of skills: application development and AI evaluation. GenAI app development is much like developing non-AI applications, requiring software skills that depend on the type of application. However, evaluation for GenAI applications requires specialized tools and techniques to handle the complexity and open-ended responses from GenAI.

To learn about building industry-specific GenAI apps on Databricks, see:

- [Databricks Solutions for Industry](#) for use cases, customers, and other

 Ask Genie

- [Databricks Solution Accelerators](#) for example use case implementations that you can run and modify

GenAI evaluation

GenAI models, agents, and applications often have complex, open-ended behavior. Users can be allowed to enter any query. An AI agent can be allowed to gather text, images, and more during execution. The output can be arbitrary text, images, or other media, and there can be many "good" answers.

These complications make it challenging to evaluate GenAI. Proper evaluation requires:

- Automation using AI to evaluate AI
- Human feedback from experts and users to collect ground-truth and calibrate automated evaluation
- Deep-diving into complex agents to understand and debug behavior

Databricks-managed MLflow and related tooling provide the foundations for GenAI evaluation:

- [Evaluate and monitor AI agents](#) – Learn about evaluation, production monitoring, and human feedback.
- [Get started: MLflow 3 for GenAI](#) – Try tracing, evaluation, and collecting human feedback.
- [MLflow Tracing – GenAI observability](#) – Learn about MLflow Tracing to record and analyze agent behavior.

Agents

An agent or agent system is an **AI-driven system that can autonomously perceive, decide, and act in an environment to achieve goals**. Unlike a standalone GenAI model that only produces an output when prompted, an agent system possesses a degree of **agency**. Modern AI agents use a GenAI model as the "brain" of a system that:

1. Receives user requests or messages from another agent.
2. Reasons about how to proceed: which data to fetch, which logic to apply, which tools to call, or whether to request more input from the user.

3. Executes a plan and possibly calls multiple tools or delegates to sub-agents.
4. Returns an answer or prompts the user for additional clarification.

By bridging **general intelligence** (the GenAI model's pretrained capabilities) and **data intelligence** (the specialized knowledge and APIs specific to your business), agent systems enable high-impact enterprise use cases such as advanced customer service flows, data-rich analytics bots, and multi-agent orchestration for complex operational tasks.

There is a continuum from simple GenAI models to complex agents. To learn more, see [Agent system design patterns](#).

Databricks provides a range of options for building agents, from fully guided to fully custom:

- [Knowledge Assistant](#) and [AI functions](#) provide guided agent builders.
- [AI Playground](#) provides a UI for prototyping tool-calling agents, from which you can export generated agent code.
- [Custom agents](#) allows you to build and deploy agents using custom code or third-party agent authoring libraries.

Tools

AI agents can call tools to gather information or perform actions. Tools are single-interaction functions that an LLM can invoke to accomplish a well-defined task. The AI model typically generates parameters for each tool call, and the tool provides a straightforward input-output interaction.

Common tool categories include:

- **Tools that retrieve or analyze data**
 - *Semantic retrieval*: Query a vector index to locate relevant text or other unstructured data.
 - *Structured retrieval*: Run SQL queries or use APIs to retrieve structured information.
 - *Web search tool*: Search the internet or an internal web corpus.
 - *Classic ML models*: Invoke machine learning models to perform classification, regression, or other predictions.

- *GenAI models*: Generate specialized outputs such as code or images.
- **Tools that modify the state of an external system**
 - *API call*: Call CRM endpoints, internal services, or other third-party integrations.
 - *Email or messaging app integration*: Post a message or send a notification.
- **Tools that run logic or perform a specific task**
 - *Code execution*: Run user-supplied or LLM-generated code in a sandbox.

Tools can be built into agentic logic or accessed using standardized interfaces like [MCP](#).

Tools vs. agents:

- Tools perform a single, well-defined operation. Agents can perform more open-ended tasks.
- Tools are generally stateless and do not maintain ongoing context beyond each invocation. Agents maintain state as they iteratively solve tasks.

Tool error handling and safety:

Because each tool call is an external operation such as an API call, the system should handle failures gracefully. Time-outs, malformed responses, or invalid inputs should not cause the agent itself to fail completely. In production, limit the number of allowed tool calls, have a fallback response if tool calls fail, and apply guardrails to ensure the agent system does not repeatedly attempt the same failing action.

Learn more about AI tools in Databricks:

- [AI agent tools](#) – Guide to tool approaches
- [Model Context Protocol \(MCP\) on Databricks](#) – Managed, custom, and external MCP servers

GenAI models and LLMs

Large Language Models (LLMs) are AI models trained on massive text data sets that can understand, generate, and reason about human language. LLMs power applications

like chatbots, code assistants, and content generation tools by predicting and producing contextually relevant text based on input prompts.

More generally, GenAI models or foundation models are trained on massive text, image, video, audio, or other data in order to learn about modes beyond text. Multi-modal models learn to connect human language with images, audio, and other media. LLMs are a type of GenAI or foundation model, though these terms are often used loosely and interchangeably.

GenAI models provide the intelligence behind GenAI agents and apps. Simple apps are often built using a single model customized with prompt engineering.

Learn about using GenAI models on Databricks:

- [Get started: Query LLMs and prototype AI agents with no code](#)
- [Databricks-hosted foundation models available in Foundation Model APIs](#)
- [Learn about AI Gateway for governing GenAI models](#)

Prompt engineering

GenAI models generally take prompts, or instructions telling the model how to handle user input. Prompts can be heavily customized with detailed steps, expert knowledge, data, and other information.

Databricks provides flexible ways to do prompt engineering. For example:

- [Knowledge Assistant](#) automates prompt engineering under the hood, based on your data and feedback.
- [AI Playground](#) provides a UI for manual, interactive prompt engineering.
- [MLflow Prompt Optimization](#) and [DSPy](#) provide data-driven prompt optimization routines.

Fine-tuning models

GenAI models can also be customized through fine-tuning, which customizes a model for a specific domain or application using your custom data.

Databricks provides flexible ways to fine-tune models. For example:



- [Knowledge Assistant](#) automates data-driven customization, with the potential to do fine-tuning under the hood.
- [AI Runtime](#) provides compute infrastructure for fully customizable fine-tuning. [See examples.](#)

What is a GenAI platform?

GenAI requires a combined data + AI platform. For both developers and administrators, the key components for GenAI must be connected and governed in a simple, unified platform.

Key components include:

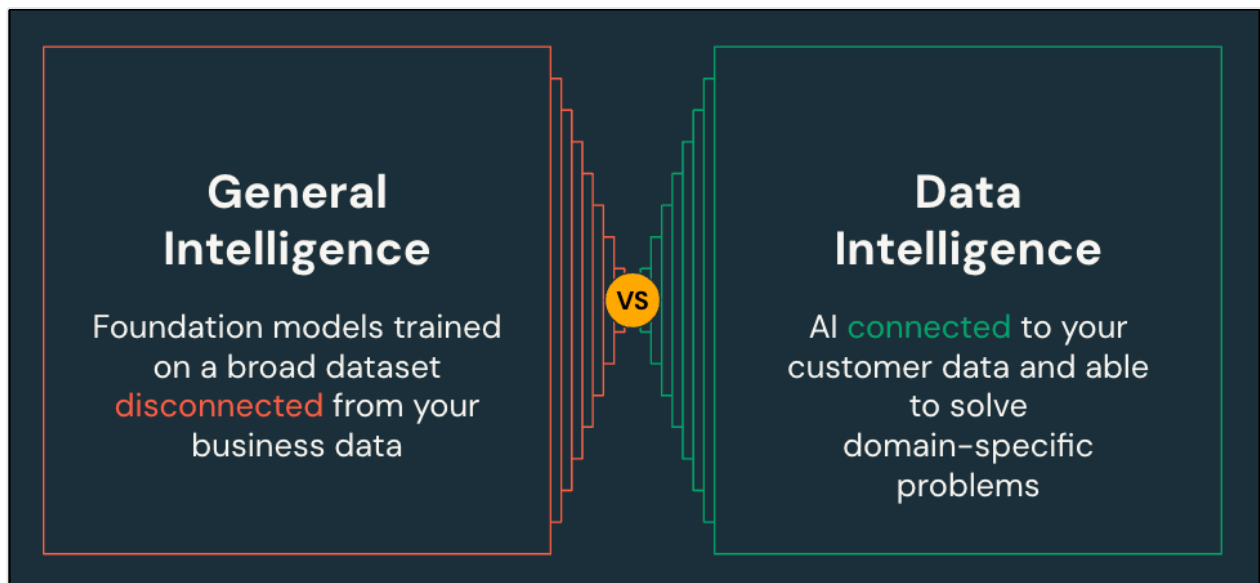
- AI assets such as models, agents, and apps
- Data assets such as files, tables, processing pipelines, vector indexes, and feature stores
- AI deployments such as endpoints for models and agents
- Tooling for building and deploying AI and data assets

Key governance capabilities include:

- Unified governance of AI and data assets. Learn more at [What is Unity Catalog?](#).
- Unified governance of GenAI model endpoints. Learn more at [Unity AI Gateway](#).
- Unified security approach. Learn more at [Databricks AI Security](#).
- Unified administration of AI and data tooling. Learn more at [Administration](#).

Also see [Databricks generative AI capabilities](#) and [Databricks architecture](#).

General intelligence vs. data intelligence



- **General intelligence** refers to what the LLM inherently knows from broad pretraining on diverse text. This is useful for language fluency and general reasoning.
- **Data intelligence** refers to your organization's domain-specific data and APIs. This might include customer records, product information, knowledge bases, or documents that reflect your unique business environment.

Agent systems blend these two sources of knowledge: They start with an LLM's broad, generic knowledge and then bring in real-time or domain-specific data to answer detailed questions or perform specialized actions. With Databricks, you can embed data intelligence into your GenAI apps at every level:

- Data sources like [vector indexes](#) and [Genie](#)
- Agents, including both [custom agent designs](#) and automated designs from [Knowledge Assistant](#) and [AI functions](#).
- [Evaluation data and metrics](#)
- [Prompt optimization](#) based on evaluation data
- Model fine-tuning, including both [custom fine-tuning](#) and automated tuning by [Knowledge Assistant](#) and [AI functions](#).

GenAI vs. ML vs. deep learning

The boundaries between generative artificial intelligence (GenAI), machine learning (ML), and deep learning (DL) can be fuzzy. This guide focuses on GenAI, but the  Ask Genie

following Databricks platform features support ML, deep learning, and GenAI:

- [Model Serving](#) supports ML, deep learning, and GenAI models. You might use it for [GenAI batch inference](#) and to deploy agents or fine-tuned models using [custom model serving](#).
- [AI Runtime](#) and GPU-enabled [Databricks Runtime for Machine Learning](#) can be used to train and fine-tune ML, deep learning, and GenAI models.
- [MLflow experiment tracking](#) can be used to track both classic ML and GenAI experiments and runs.
- [Databricks Feature Store](#) can be used to manage and serve structured data for both classic ML and GenAI.

Learn more

- [Key challenges in building GenAI apps](#) – Major challenges in GenAI, and solutions to them using Databricks
- [Agent system design patterns](#) – Simple to complex GenAI agents, plus practical advice
- [AI on Databricks](#) – Use cases, customers, and other resources for AI on Databricks

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 6, 2026**

Key challenges in building GenAI apps

Despite the power of modern GenAI models, production-grade generative AI applications are often challenging to build. Three key challenges can be summarized as:

- **Governance:** Many platforms struggle to provide unified governance, data privacy, and security for data and AI assets.
- **Quality:** The flexible and unpredictable behavior of GenAI models adds complexity to evaluation.
- **Control:** Platforms must provide flexibility, model choice, and customization.

Governance for data and AI

GenAI applications require diverse data and AI assets: tables, vector indexes, AI models, tools, and more. A GenAI platform must provide fine-grained access to these assets to developers, while providing joint governance for administrators. Without complete governance, organizations face risks such as:

- **Data leakage:** Sensitive customer or enterprise data can be misused without proper lineage tracking and access control, and data can inadvertently leak through model outputs if proper guardrails are not enforced.
- **Compliance restrictions:** Many organizations have compliance requirements such as SOC2 or HIPAA, and integrating GenAI models into compliant legacy platforms can be complex, leading to delays or restrictions in using the best models.
- **Unauthorized usage or unexpected costs:** Without access controls and usage guardrails, AI models might be used by unauthorized teams or incur high usage costs.

Databricks simplifies unified governance for data and AI through:

- [Unity Catalog](#), which manages files, tables, vector indexes, feature stores, models, and tools under a unified governance model
- [AI Gateway](#), which provides unified governance and monitoring for AI model endpoints, including safety guardrails and usage limits
- [Databricks AI Security Framework](#), which provides a comprehensive guide to AI risk management
- [Databricks AI Governance Framework](#), which complements the Security Framework by providing a view of governance spanning both security and operational integrity

Quality of models, agents, and apps

GenAI models produce open-ended, stochastic outputs and are often applied to open-ended problems with many "good" answers. Even defining "high quality" can be challenging and often requires iterative feedback from domain experts or users.

Without robust evaluation processes, organizations face risks such as:

- **Bad user experiences:** If GenAI apps are not evaluated based on metrics aligned with user needs, then users can find responses unhelpful, inaccurate, or even harmful or offensive. Brand reputations can suffer in extreme cases.
- **Development limbo:** If quality cannot be defined or measured in ways allowing stakeholder sign-off, GenAI projects can be delayed or canceled for lack of "proof" of quality.

Databricks simplifies measuring and optimizing AI quality through:

- [MLflow Evaluation and Monitoring](#), with built-in judges and custom scorers to measure quality, usable in both development and production monitoring
- [MLflow Tracing](#), with both automatic and manual tracing to provide observability for both development and production
- [Human feedback collection](#), with a built-in app for expert feedback during development and APIs for user feedback from production apps
- Methods for optimizing trade-offs between quality, cost, and latency. [Knowledge Assistant](#) provides a guided optimization experience for QA chatbots, and [Ask Genie](#)

[Framework](#) and a [flexible choice of AI models](#) provide trade-off options for fully custom agents.

Control of data and models

State-of-the-art GenAI models are provided by many model providers, as well as self-hosted open-source options. Because of data privacy and licensing complications, many platforms struggle to support this diverse ecosystem and to allow fast iteration and customization. Organizations must maintain control of their data and choice over models to avoid risks such as:

- Data privacy restrictions: Compliance or integration requirements can prevent organizations from accessing the top GenAI models from multiple providers, sacrificing flexibility and quality-cost trade-offs.
- Lack of competitive edge: If models, data, agents, and applications are not customizable based on an organization's proprietary data, then it is challenging to build intellectual property.

Databricks provides control and flexibility for data and models through:

- [Foundation Model APIs](#), which serve frontier models from top model providers in your own Databricks environment, alongside your custom models and agents in [Model Serving](#).
- Customized apps, agents, models, tools, and data sources built around your proprietary data. All of these levels of AI support [Data Intelligence](#), from building apps and agents, to providing data through tools, to evaluating and optimizing agents based on your data.

Learn more

- [Databricks generative AI capabilities](#)
- [Open source vs. managed MLflow on Databricks](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 8, 2026**

Agent system design patterns

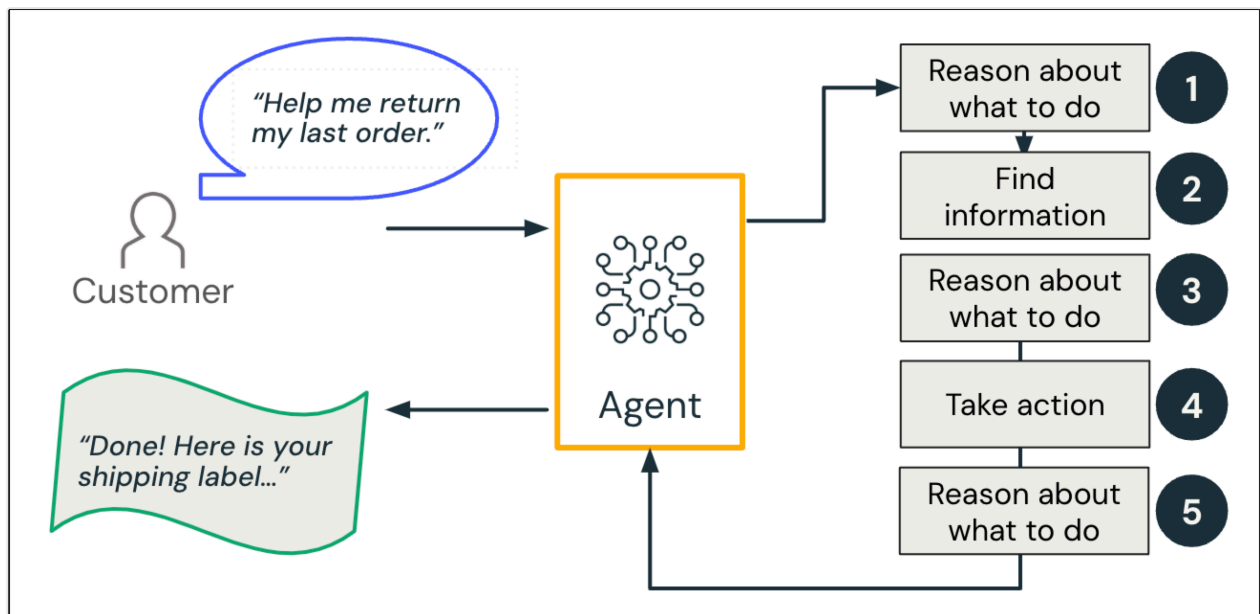
GenAI agents combine the intelligence of GenAI models with tools for data retrieval, external actions, and other capabilities. This page walks through agent design:

- [A concrete example](#) of building an agent system illustrates orchestrating how model and tool calls flow together.
- [Design patterns for agent systems](#) form a continuum of complexity and autonomy, from deterministic chains, through single-agent systems that can make dynamic decisions, up to multi-agent architectures that coordinate multiple specialized agents.
- [A practical advice section](#) gives advice on choosing the right design and on developing agents, testing, and moving to production.

Agents rely heavily on tools for gathering information and taking external actions. For more background on tools, see [Tools](#).

Example agent system

For a concrete example of an agent system, consider a call center GenAI agent interacting with a customer:

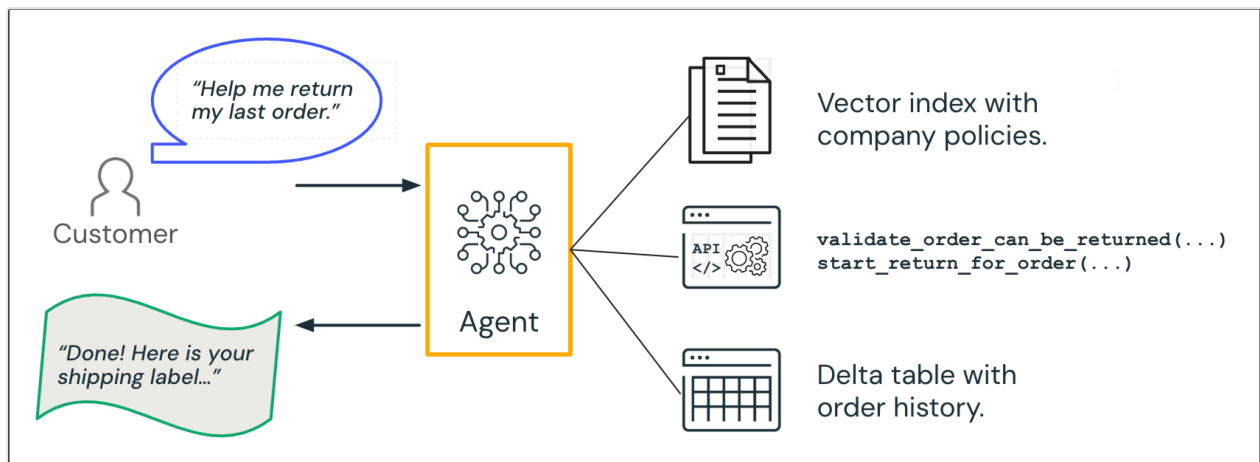


The customer makes a request: "Can you help me return my last order?"

1. **Reason and plan:** Given the intent of the query, the agent "plans": "Look up the user's recent order and check our return policy."
2. **Find information** (data intelligence): The agent queries the order database to retrieve the relevant order and references a policy document.
3. **Reason:** The agent checks whether that order fits in the return window.
 - **Optional human-in-the-loop:** The agent checks an additional rule: If the item falls into a certain category or is outside the normal return window, escalate to a human.
4. **Action:** The agent triggers the return process and generates a shipping label.
5. **Reason:** The agent generates a response to the customer.

The AI agent responds to the customer: "Done! Here is your shipping label..."

These steps are second nature in a *human* call center context. In an *agent system* context, the LLM "reasons" while the system calls on specialized tools or data sources to fill in the details.



Levels of complexity: From LLMs to agent systems

GenAI apps can be powered by a range of systems, from simple LLM calls to complex multi-agent systems. When building any AI-powered application, start simple. Introduce more complex agentic behaviors when you truly need them for better flexibility or model-driven decisions. Deterministic chains offer predictable, rule-based flows for well-defined tasks. More agentic approaches offer greater flexibility and potential, but they come with the cost of extra complexity and potential latency.

Design Pattern	When to Use	Pros	Cons
LLM + prompt	- Generic question and answer - Quick prototype for short-term use	- Very simple - Easy to create	Minimal customization
Deterministic chain	- Well-defined tasks - Static pipelines such as basic RAG - No need for on-the-fly decisions	- Simple - Easy to audit	- Inflexible - Requires code changes to adapt

Design Pattern	When to Use	Pros	Cons
Single-agent system	- Moderate to complex queries in the same domain - Some dynamic decisions without the overhead of multiple specialized agents	- Flexible - Simpler than multi-agent - Good "default"	- Less predictable - Must guard against repeated or incorrect tool calls
Multi-agent system	- Large or cross-functional domains - Multiple "expert" agents, with distinct logic or conversation contexts	- Highly modular - Scales to big domains	- Complex to orchestrate - Harder to trace and debug

[Agent Framework](#) is agnostic to these patterns, making it easy to start simple and evolve toward higher levels of automation and autonomy as your application requirements grow.

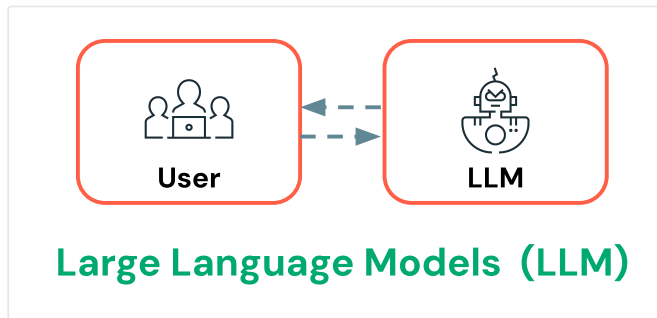
To read more about the theory behind agent systems, see blog posts from the Databricks founders:

- [AI agent systems: Modular Engineering for Reliable Enterprise AI Applications](#)
- [The Shift from Models to Compound AI Systems](#)

LLM and prompt

The simplest design has a standalone LLM or other GenAI model that responds to prompts based on knowledge from a vast training dataset. This design is good for simple or generic queries but is often disconnected from your real-world business

data. You can customize behavior by providing a system prompt with your custom instructions or embedded data.

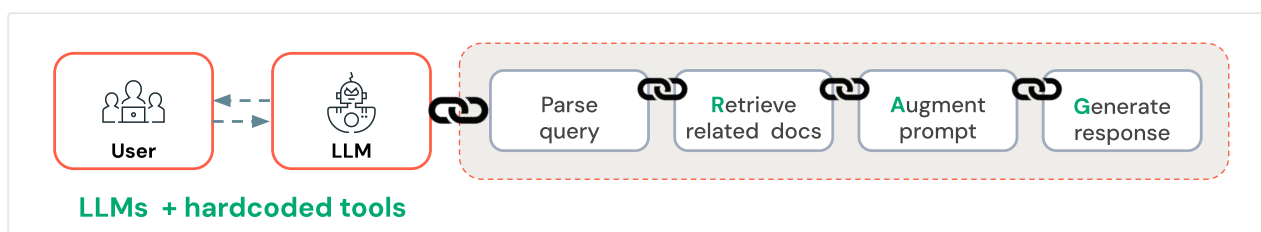


Deterministic chain (hard-coded steps)

Deterministic chains augment GenAI models with tool calling, but the developer defines which tools or models are called, in what order, and with which parameters. The LLM does not make decisions about *which tools* to call or in *what order*. The system follows a predefined workflow or "chain" for all requests, making it highly predictable.

For example, a deterministic Retrieval Augmented Generation (RAG) chain might always:

1. Retrieve top-k results from a vector index to find context relevant to a user request.
2. Augment a prompt by combining the user request with the retrieved context.
3. Generate a response by sending the augmented prompt to an LLM.



When to use:

- For well-defined tasks with predictable workflows.
- When consistency and auditing are top priorities.
- When you want to minimize latency by avoiding multiple LLM calls for orchestration decisions.

Advantages:

- Highest predictability and auditability.
- Typically lower latency (fewer LLM calls for orchestration).
- Easier to test and validate.

Considerations:

- Limited flexibility for handling diverse or unexpected requests.
- Can become complex and difficult to maintain as logic branches grow.
- Can require significant refactoring to accommodate new capabilities.

Single-agent system

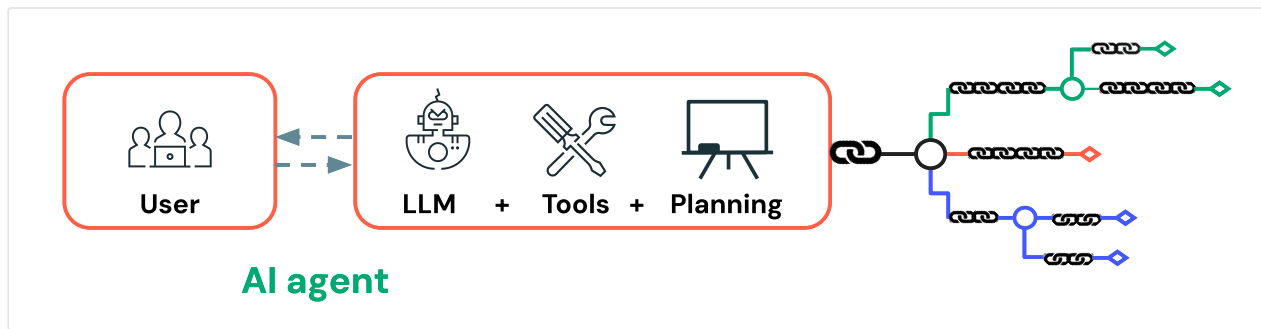
A single-agent system has an LLM that orchestrates one coordinated flow of logic. The LLM adaptively decides which tools to use, when to make more LLM calls, and when to stop. This approach supports dynamic, context-aware decisions.

A single-agent system can:

1. Accept requests such as user queries and any relevant context such as conversation history.
2. Reason about how best to respond, optionally deciding whether to call tools for external data or actions.
3. Iterate if needed, calling an LLM or tools repeatedly until an objective is achieved or a certain condition is met, such as receiving valid data or resolving an error.
4. Integrate tool outputs into the conversation.
5. Return a cohesive response as output.

For example, a help desk assistant agent might adapt as follows:

- If the user asks a simple question ("What is our returns policy?"), the agent might respond directly from the LLM's knowledge.
- If the user wants their order status, the agent might call a function `lookup_order(customer_id, order_id)`. If that tool responds with "invalid order number," the agent can retry or prompt the user for the correct ID, continuing until it can provide a final answer.



When to use:

- You expect varied user queries but still within a cohesive domain or product area.
- Certain queries or conditions can warrant tool usage such as deciding when to fetch customer data.
- You want more flexibility than a deterministic chain but don't require separate specialized agents for different tasks.

Advantages:

- The agent can adapt to new or unexpected queries by choosing which (if any) tools to call.
- The agent can loop through repeated LLM calls or tool invocations to refine results – without needing a fully multi-agent setup.
- This design pattern is often the sweet spot for enterprise use cases – simpler to debug than multi-agent setups while still allowing dynamic logic and limited autonomy.

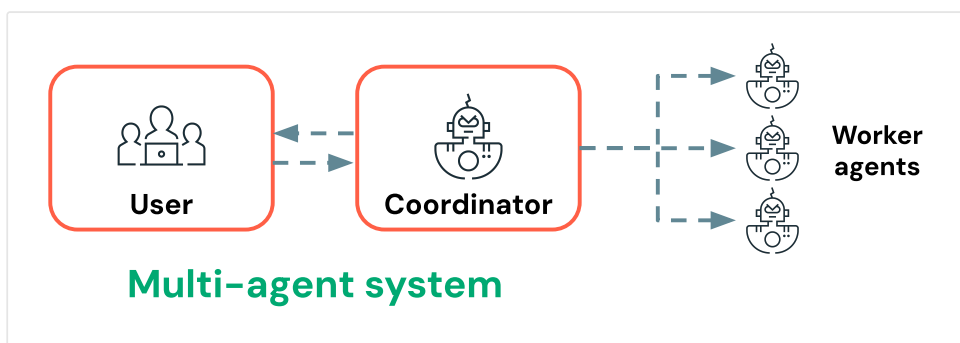
Considerations:

- Compared to a hard-coded chain, you must guard against repeated or invalid tool calls. Infinite loops can occur in any tool-calling scenario, so set iteration limits or timeouts.
- If your application spans radically different sub-domains (finance, devops, marketing, etc.), a single agent can become unwieldy or overloaded with functionality requirements.
- You still need carefully designed prompts and constraints to keep the agent focused and relevant.
- Agency is a continuum; the more freedom you provide models to control the behavior of the system, the more agentic the application becomes. In practice, it's a spectrum between a simple tool and a fully autonomous agent.

most production systems carefully constrain the agent's autonomy to ensure compliance and predictability, for example by requiring human approval for risky actions.

Multi-agent system

A multi-agent system involves two or more specialized agents that exchange messages or collaborate on tasks. Each agent has its own domain or task expertise, context, and potentially distinct tool sets. A separate "coordinator" or "AI supervisor" directs requests to the appropriate agent, or decides when to hand off from one agent to another. The supervisor can be another LLM or a rule-based router.



For example, a customer assistant might have a supervisor that delegates to specialized agents:

- Shopping assistant: Helps customers search for products and provides advice on pros and cons from reviews
- Customer support agent: Handles feedback, returns, and shipping

When to use:

- You have distinct problem areas or skill sets such as a coding agent or a finance agent.
- Each agent needs access to conversation history or domain-specific prompts.
- You have so many tools that fitting them all into one agent's schema is impractical; each agent can own a subset.
- You want to implement reflection, critique, or back-and-forth collaboration among specialized agents.

Advantages:

- This modular approach means each agent can be developed or maintained by separate teams, specializing in a narrow domain.
- Can handle large, complex enterprise workflows that a single agent might struggle to manage cohesively.
- Facilitates advanced multi-step or multi-perspective reasoning – for instance, one agent generating an answer, another verifying it.

Considerations:

- Requires a strategy for routing between agents, plus overhead for logging, tracing, and debugging across multiple endpoints.
- If you have many sub-agents and tools, it can get complicated to decide which agent has access to which data or APIs.
- Agents can bounce tasks indefinitely among themselves without resolution if not carefully constrained. Infinite loop risks also exist in single-agent tool-calling, but multi-agent setups add another layer of debugging complexity.

Practical advice

If your use case can be addressed using [Knowledge Assistant](#) or an [AI functions](#) offering, then start with that guided, simpler option.

If you need to build a custom agent system, then Databricks and the [Agent Framework](#) are agnostic of whatever pattern you choose, making it easy to evolve design patterns as your application grows. Consider the following best practices for developing stable, maintainable agent systems:

1. **Start simple:** If you only need a straightforward chain, a deterministic chain is fast to build.
2. **Gradually add complexity:** As you need more dynamic queries or flexible data sources, move to a single-agent system with tool calling. If you have clearly distinct domains or tasks, multiple conversation contexts, or a large tool set, then consider a multi-agent system.
3. **Combine patterns:** In practice, many real-world agent systems combine patterns. For instance, a mostly deterministic chain can have one step in which the LLM can dynamically call certain APIs if needed.

Development guidance

- **Prompts and tools**
 - Keep prompts clear and minimal to avoid contradictory instructions, distracting information and reduce hallucinations.
 - Provide only the tools and context your agent requires, rather than an unbounded set of APIs or large irrelevant context. [Choose your tool approach](#) during design.
- **Logging and observability**
 - Implement detailed logging for each user request, agent plan, and tool call using [MLflow Tracing](#).
 - Store logs securely and be mindful of personally identifiable information (PII) in conversation data. Consider [Data Classification](#) for automation.

Testing and iteration guidance

- **Evaluation**
 - Use [MLflow Evaluation](#) and [production monitoring](#) to define evaluation metrics for development and production.
 - Gather [human feedback](#) from experts and users to ensure your automated evaluation metrics are well-calibrated.
- **Error handling and fallback logic**
 - Plan for tool or LLM failures. Timeouts, malformed responses, or empty results can break a workflow. Include retry strategies, fallback logic, or a simpler fallback chain when advanced features fail.
- **Iterative improvements**
 - Expect to refine prompts and agent logic over time. Version changes using the [MLflow Prompt Registry](#) for your prompts. Versioning will simplify operations and allow rollbacks and comparisons.
 - As you collect evaluation data and define metrics, consider more automated optimization methods such as [MLflow Prompt Optimization](#).

Production guidance

- **Model updates and version pinning**

- LLM behaviors can shift when providers update models behind the scenes. Use version pinning and frequent regression tests to ensure your agent logic remains robust and stable.

- **Latency and cost optimization**

- Each additional LLM or tool call increases token usage and response time. Where possible, combine steps or cache repeated queries to keep performance and cost manageable.

- **Security and sandboxing**

- If your agent can update records or run code, sandbox those actions or enforce human approval where necessary. This is critical in enterprise or regulated environments to avoid unintended harm. [Unity Catalog functions](#) provide sandboxed execution for production.
- See [AI agent tools](#) for more guidance on tool options.

By following these guidelines, you can mitigate many of the most common failure modes such as tool mis-calls, drifting LLM performance, or unexpected cost spikes, and build more reliable, scalable agent systems.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback



Last updated on **May 6, 2026**

Databricks generative AI capabilities

Databricks provides a platform for building, evaluating, deploying, and monitoring generative AI applications (GenAI apps). It brings together a suite of tools that tackle the [challenges of developing enterprise-grade GenAI apps](#). Databricks [integrates with popular open source frameworks](#), adding enterprise-grade governance, observability, and operational tooling, collectively known as LLMOps.

This page lists major features for GenAI, organized by GenAI workflow stages.

Query GenAI

Databricks makes state-of-the-art GenAI models from top model providers readily available through [Databricks-hosted Foundation Models](#). You can also query models from [external providers](#) and your [custom models](#). All of these models can be queried through UI, API/SDK, and SQL interfaces. This optionality lets you use GenAI for all use cases---general chat, complex agents, automated data pipelines, interactive data analytics, and more.

Method	Features
UI	• For development, AI Playground provides a UI for querying available GenAI models and agents. • For testing, your agents and apps can be queried and evaluated by domain experts using the Review App. • For production, your apps can be hosted with a UI using Databricks Apps for use inside your organization. For external apps, you can power user-facing apps using Databricks APIs.

Method	Features
API and SDK	• Model Serving provides REST API endpoints for querying models and agents. • See more query options including the OpenAI client and the Databricks Python SDK.
SQL	• AI functions provide task-specific and general-purpose SQL functions for querying models and agents.

Build GenAI

Databricks provides a flexible set of tools for building GenAI apps, agents, tools, and models. These include UI and code-based frameworks, all of which can optimize GenAI systems based on your data. You can leverage any open-source GenAI framework and can integrate custom tools and MCP servers.

Category	Features
Apps	• Databricks Apps provide flexible app development and deployment to authenticated users. • Custom apps can be powered by Databricks APIs.
Agents	• Knowledge Assistant provides guided agent development and optimization. • AI Playground provides a UI for prototyping tool-calling agents. • Custom code and open-source frameworks can be developed and deployed using Agent Framework or Databricks Apps.
Tools	• Tool support includes MCP servers and Unity Catalog Functions, both of which provide Unity Catalog-based governance.
 Ask Genie	

Category	Features
Models and prompts	- Prompt engineering can be done interactively using AI Playground, or through data-driven optimization using MLflow Prompt Optimization. - Fine-tune models using AI Runtime.

Prepare and serve data

Databricks simplifies data for AI by unifying governance of traditional data and AI workloads. With all data managed under [Unity Catalog](#) with fine-grained access controls, it is easy to adjust data engineering and AI boundaries to fit your organization. Data can be prepared for GenAI using any [data engineering tools](#) such as [Lakeflow Spark Declarative Pipelines](#). A table in Unity Catalog can be served for GenAI, using a vector index for unstructured data or a feature table for structured data.

Type of data	Features
Unstructured (text, images, etc.)	Vector Search automatically indexes your knowledge base at scale for semantic or hybrid search.
Structured (tables)	Serverless SQL lets you integrate tables and SQL queries into your GenAI app for analytics or transformations. Genie agents can be used in multi-agent systems to answer natural language queries about your structured data. Online feature stores provide real-time feature access for your GenAI app.

Deploy and serve GenAI

Databricks provides production-ready serving systems for GenAI apps, agents, and models backed by [Databricks Apps](#). These scalable deployments can be used for both

real-time serving and [batch inference](#). All deployments integrate with [observability](#) and [evaluation and monitoring](#) tooling.

Development framework	Deployment and serving
Databricks Apps	Apps and agents can be deployed using Databricks Apps, which provides UI- and API-based deployment .
Knowledge Assistant	Knowledge Assistant automates deployment of your agent to Model Serving endpoints.

Trace, evaluate, and monitor GenAI

Databricks provides [managed MLflow for GenAI observability, evaluation, and monitoring](#). Open-source APIs make integration and portability simple, while the managed service provides production-ready endpoints. Databricks-managed MLflow can be used for GenAI apps and agents hosted on Databricks and hosted elsewhere.

Category	Features
Tracing and observability	• MLflow Tracing allows you to instrument your GenAI agents and apps to collect telemetry and observability data for evaluation, production monitoring, and auditing.
Evaluation	• MLflow Evaluation lets you measure the quality of GenAI systems. • Built-in and custom LLM judges and scorers and evaluation datasets let you tailor evaluation to your use case.
Monitoring	• Production monitoring lets you measure quality on production traces, using the same judges and scorers from development-time evaluation.
 Ask Genie	

Category	Features
Human feedback	- During development, the Review App lets you run quick vibe checks using a Chat UI and collect expert feedback in labeling sessions. - During production, APIs let you annotate traces with user feedback.

LLMOps

Databricks provides a full suite of tools for GenAI operations, or "LLMOps." Unified governance of data and AI assets under [Unity Catalog](#) simplifies and secures deployment of AI across an organization. AI Gateway simplifies managing models from many GenAI model providers. MLflow and Declarative Automation Bundles provide versioning and infrastructure-as-code for implementing robust LLMOps processes.

Category	Features
Data and AI asset governance	Unity Catalog provides unified governance for data and AI assets. Data assets include files, tables, vector indexes, and feature stores. AI assets include models, tools, and connections for MCP servers and other APIs.
Model endpoint governance	AI Gateway provides central governance and monitoring for GenAI model endpoints.
Prompt versioning	MLflow provides a Prompt Registry , as well as experiment tracking.
Infrastructure as code	MLOps Stacks , which is built on top of Databricks Assets Bundles , provides code-based management and deployment of infrastructure and workflows.

Open source support

Databricks provides full support for the rapidly growing open-source ecosystem for GenAI.

For development, you can use any open-source framework and deploy it using [Databricks Apps](#). Databricks services can be used by third-party GenAI tools and apps by using [MCP servers](#) or the [REST API or SDKs](#).

For observability, evaluation, and monitoring, [MLflow Tracing](#) provides native autologging for [20+ open-source GenAI frameworks](#), and you can add custom tracing to any other frameworks or code. The traces can then be used with [MLflow Evaluation and production monitoring](#). The traces follow [OpenTelemetry trace specs](#) and can be exported to third-party tools.

Learn more

- [AI and machine learning tutorials](#)
- [Open source vs. managed MLflow on Databricks](#)

Is this page

as this page helpful?

☐ Yes

☐ No



Last updated on **May 6, 2026**

Guide: Agents development workflow

This guide provides a starting point to understand the full lifecycle of building an AI application or AI agent. Throughout this guide, "AI agent" is an umbrella term for GenAI-powered systems, including simple LLM calls, AI functions, and agent-based implementations.

Overview of the development lifecycle

1. [Understand use case, scope, and success metrics](#)
2. [Build an initial AI agent](#)
3. [Iterate on AI agent quality](#)
4. [Align with stakeholders before production](#)
5. [Release to production and continuously monitor quality](#)

1. Understand use case, scope, and success metrics

Before building anything, clarify what the AI agent is meant to do. Align with stakeholders, including the people who will sign off on deploying to production.

- What types of inputs will the agent handle (the "domain" or "scope")? What users will submit the inputs?
- How should the agent ideally respond to common inputs? What information or context should it use?
- What criteria define a good or bad response: tone, accuracy, completeness, response length, safety, citations, or other requirements?



Ask Genie

- What system requirements and constraints are there in production: cost, latency, and scalability?
- What are potential failure modes, and how should the agent handle them: bad user inputs, insufficient information to answer, user feedback indicating a bad answer, or others?

Choose the simplest viable approach. Many use cases do not require complex agentic or multi-agent systems. Before building, assess where your problem falls on the complexity continuum. Will simple deterministic logic or batch AI functions suffice? If dynamic tool-calling, reasoning, or coordination are needed, then consider tool-calling agents or multi-agent systems. For deeper guidance, see the [Agent system design patterns](#).

This foundation enables you to:

1. Identify the data sources and tools your agent will need
2. Write initial instructions or prompts that reflect the intended behavior
3. Identify domain experts or testers who can provide representative examples and early feedback
4. Create automated judges that encode the assessment criteria and accelerate iteration

You do not need perfect clarity at this stage, and your understanding will improve as you iterate. But stronger early alignment, especially on how quality will be measured and what "production ready" means, makes later quality improvements and sign-off significantly faster.

2. Build an initial AI agent

After your use case and goals are well-defined, you are ready to prototype your AI agent. Databricks provides both guided, UI-based routes and fully custom, code-based routes for building AI agents.

2.1. Prepare data and tools

AI agents generally use data and tools to provide context and abilities. See [AI agent tools](#) for an overview of working with data and tools on Databricks.

Search for existing data and tools before creating new ones:

- Explore available data in [Unity Catalog](#) or [workspace search](#) to understand what governed assets already exist. This helps you understand what context and capabilities are available before creating new assets.
- In [AI Playground](#), you can view and select tools that are already available to agents, such as Vector Search indexes, MCP servers, or UC Functions.

Create and manage new assets as needed:

- Prepare and serve [structured data](#) or [unstructured data](#).
- Create simple or complex tools using [managed or external MCP servers](#).

All of these data assets and tools are governed and versioned in Unity Catalog, making them discoverable and reusable across AI agents and applications.

2.2. Build an initial agent

Before building a custom agent, assess whether a declarative [Knowledge Assistant](#) agent, [AI functions](#), or an existing [Databricks solutions accelerator](#) already matches your use case. For common patterns, these guided approaches can significantly reduce setup, improve default quality, and speed time to production.

If a custom agent is still required, new builders should start with the fastest way to experiment. Use [AI Playground to prototype an agent](#) without writing code. AI Playground allows you to try different models, do prompt engineering, and test tools in order to understand data quality, agent behavior, and the potential of your approach quickly. You can then export the agent as code for further customization and iteration.

If you already have agent code, you can [bring existing code into Databricks and deploy it as a Databricks App](#).

As you build your agent, plan ahead for evaluation and production:

- Instrument your agent with [MLflow Tracing](#) to record and analyze agent behavior.
 - At this stage, focus on functional correctness: make sure the agent runs end to end and can access the required data and tools.
 - Vibe check for early issues such as wrong tool selection, missing context, or hallucinations.

- Later, these traces will be used for evaluating agent quality.
- During implementation, consider the [right authentication method for your production application](#).

3. Iterate on AI agent quality

After a working prototype exists, the next phase is a tight loop of measuring, understanding, and improving quality. Databricks places [MLflow Evaluation](#) at the center of this loop, supported by MLflow Tracing, evaluation datasets, and LLM judges.

Automated scorers and LLM judges provide scale and consistency, but [human feedback](#) is critical for validating real world usefulness and understanding subtle failures. Human feedback also guides development and calibration of LLM judges. Human feedback typically enters in three stages as the agent matures:

1. Early developer and stakeholder validation
2. Broader domain experts review
3. End user feedback

3.1. Validate early behavior

Developers and a small group of stakeholders or domain experts can provide quick, early feedback. Before scaling testing and evaluation, confirm the agent does the right things in the most obvious situations.

During prototyping, developers often perform an informal "vibe check" by manually querying the agent to confirm it runs end to end and behaves as expected. With the MLflow Tracing UI, [developers can attach feedback or expectations](#) directly to traces to flag quality issues, mark successful examples, and capture notes for future evaluation and iteration.

After you deploy an internal prototype, the [Review App Chat UI](#) provides a simple UI for collecting feedback. Share the Chat UI for your prototype with a small set of developers or domain experts who can ask both reasonable and problematic queries.

MLflow Tracing records the interactions and feedback to build an initial dataset of results. [Analyze traces with the MLflow UI or code](#) to understand the agent's  Ask Genie

performance and behavior. If results are bad or unexpected, use the traces to debug:

- Analyze quality issues in the agent, such as tool misuse, hallucinations, or missing context. Apply fixes, such as prompt tuning, tool usage, or data. See [3.4. Fix issues and re-verify improvements](#).
- As you iterate, you can use the trace dataset as representative user inputs to generate traces for your new prototype.
- Repeat this loop: run, inspect, fix, and re-run, until the agent handles all or most of the representative inputs as expected.
- More issues may be uncovered and addressed in later iterations. Quality improvement is iterative and not limited to this early phase.

After this step, you can feel confident the prototype behaves sensibly in common cases and achieves a reasonable level of quality, before investing in more extensive testing.

3.2. Expand testing and feedback

After the prototype works in simple cases, scale up quality evaluation by broadening your set of beta testers and by collecting more customized feedback. This phase reveals blind spots such as unexpected topics, misunderstood queries, tools and retrieval gaps, or emerging usage patterns. It also expands your evaluation datasets.

- Roll out the application to a broader set of stakeholders and domain experts, or to beta end users. Incorporate their feedback as the agent is exposed to broader usage patterns.
- Capture more detailed feedback and expectations using [Review App labeling sessions](#) with custom schema for expert feedback.
- Build [evaluation datasets](#) by syncing human feedback and labeled traces, preparing for systematic evaluation and monitoring in the next step.
- To further enrich the evaluation dataset, consider [generating synthetic evaluation sets](#).

3.3. Evaluate quality and debug systematically

As your evaluation datasets become larger and more diverse, you will need structured and more automated ways to detect issues, surface the most important failures, and

understand root causes.

In practice, you will likely divide your data into two types of evaluation datasets:

- **Regression testing:** Data with high-quality AI responses helps to define expected behavior. Use these datasets to validate that new versions of the agent continue to perform well across a broad and diverse set of expected scenarios.
- **Issue-focused debugging:** Data with low-quality AI responses may include a variety of unwanted behaviors. Isolate groups of traces that exhibit the same types of low-quality behavior so you can understand the root causes and iterate on targeted fixes.

The tools below help to build and analyze both types of evaluation datasets.

Run regression tests

- Build regression tests by selecting representative subsets of data for which you have high-quality AI responses or human expectations.
- Define evaluation criteria using [built-in or custom LLM judges and scorers](#). Automated evaluation can use LLMs alone to assess response quality, or they can compare responses against ground-truth responses or expectations.
- [Run evaluation](#) on new versions of your agent to ensure updates do not degrade previously good behavior.

Identify types of low-quality responses

- Use both [automated evaluation](#) and [human feedback](#) to spot examples where the agent responds poorly.
- [Filter and analyze MLflow traces](#) by judge scores or user feedback to isolate problematic interactions. With specific judges and custom feedback schema, you can isolate specific types of problems, such as hallucination, missing context, or irrelevant responses.
- For agentic debugging, you can use [MLflow AI Insights](#) or connect your own agents to the [MLflow MCP server](#).

Improve the accuracy of automated detection

Though you can begin to build evaluation datasets using mostly human feedback, you can scale evaluation with automated detection. As you iterate, invest in LLM judges or code-based scorers tailored to your application and domain.

- Start with [built-in judges](#), and add [custom judges](#) and [code-based scorers](#) as needed. When you observe a failure mode not captured by a built-in judge, you can automate future detection with a custom judge or scorer designed to detect that specific type of failure.
- [Use human feedback to align custom judges](#) with expert understanding. Tuning judges to reduce false positives and negatives will increase trust in automated evaluation and triage.
- Your new judges and scorers can be used both for automated evaluation and monitoring and for filtering traces to build datasets for debugging.

Root cause issues effectively

After a failure is identified, you need to determine why it occurred.

- [Use MLflow Tracing](#) to inspect each step of the agent's reasoning manually:
 - Which tools were selected
 - How tool inputs and outputs were used
 - Whether retrieval returned relevant context
 - How model responses influenced downstream decisions
- Apply [MLflow AI Insights](#) or [agent-as-a-judge](#) to analyze traces and point to likely causes such as poor grounding, bad prompt structure, or incorrect tool arguments.
- [Compare versions](#) in MLflow's evaluation UI to see whether issues regress or persist across iterations.

The ideal outcome of this step is to have a structured understanding of what is failing, why it fails, and how to fix it. Automation and application-specific judges allow you to iterate confidently as your agent grows more capable and the test set grows more complex.

3.4. Fix issues and re-verify improvements

Just as issues are application-specific, fixes must be tailored to your application. Examples of common fixes include:

- Prompt optimization: Refine the agent's instructions manually, or use [data-driven prompt optimization](#). For broader agent optimization such as tuning multi-step reasoning or tool use, use [DSPy tuning](#).
- Tools and data: Improve tools or retrieval flows when traces show missing facts or poor grounding.
- Routing: When traces show the wrong tools or sub-agents were called, improve tool or agent metadata, prompts, or the routing model.
- Guardrails: When responses violate safety rules or leak information, use either [AI Gateway guardrails](#) or customized guardrails in your agent.
- Fallbacks: Handle extreme cases, missing data, or API call failures gracefully using fallback mechanisms such as alternate API endpoints or fallback responses.

As you iterate on fixes, use the [Prompt Registry](#) to record versions for simpler comparisons and regression testing.

Each fix to prompts, retrieval, tools, data, or other parts of your agent should be validated the same way it was discovered. Re-run the new agent version on the same evaluation datasets to confirm that the issue is fixed and no regressions have been introduced.

4. Align with stakeholders before production

Before releasing an agent into a real environment, teams need a shared understanding of its current capabilities, limitations, and measured quality. Getting to this point typically requires multiple rounds of iteration and quality improvement in [Step 3](#). At this stage, translate the technical signals (such as evaluation metrics, system metrics, and example traces) into the business context that ultimately determines whether the agent is truly "ready."

- Translate evaluation results into clear business signals: Summarize accuracy, stability, safety, and known limitations in language stakeholders can act on.

- Confirm standardized quality checks are met: Make sure required evaluation metrics, regression checks, and dataset coverage thresholds pass for the candidate version.
- Validate operational readiness and obtain sign-off: Review the monitoring setup, guardrails, and rollout plan. Document risks and acceptance criteria before production.

5. Release to production and continuously monitor quality

Reaching production is a major milestone! It means the agent is ready for real users and real impact. At the same time, production is also the beginning of a new cycle. After an agent is live, it enters continuous monitoring and improvement because real usage will surface new behaviors, edge cases, and issues.

- [Collect feedback from end users](#) in production. Link user feedback to specific traces so it can be analyzed alongside model behavior. You can do this by logging feedback as assessments attached to the original trace.
- Leverage [AI Gateway](#) for guardrails, routing, and consistent logging. Make sure each new agent version can be evaluated against real traffic without operational friction.
- [Monitor quality on live traffic](#) by running evaluation on sampled production traces. Confirm the new version performs at least as well as prior versions, and look for new issues as users submit new types of queries. Continuous monitoring keeps the agent reliable, safe, and aligned with business needs as it evolves. MLflow provides a monitoring dashboard, but since [traces can be stored in Unity Catalog](#), you can customize dashboards and alerts:
 - Build [custom dashboards](#) for monitoring and sharing with business stakeholders.
 - Set up [Databricks SQL quality alerts](#) to detect failures or emerging issues.
- Act on production insights:
 - For high-risk use cases, link monitoring to automated or gated rollback mechanisms to fix critical issues.
 - Use your production insights in your next iteration. Convert real-world failures into new evaluation data, and return to the [evaluation and debugging loop](#) to

build the next, better version of your agent.

Next steps

- [Databricks generative AI capabilities](#) – Review the Databricks platform capabilities for agents and GenAI
- [Agent system design patterns](#) – Learn about the range from simple to complex agents
- [Get started: Query LLMs and prototype AI agents with no code](#) – Prototype an agent using AI Playground

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback